CANDY
DIALOGUE ENGINE

Visual Novels

1.1.0

If you want to make a visual novel game, Candy DE offers a wide range of tools and settings.

VN Scene

First, it's important to understand that for visual novels, Candy DE makes use of VN Scenes.

VN Scenes are UI elements and contain nodes called "bust nodes" for displaying actor busts. Actor busts are the images that represent characters in a visual novel layout.

Multiple node types can be used as bust nodes: TextureRect, TextureButton, NinePatchRect, Sprite2D/3D, AnimatedSprite2D/3D, or VideoStreamPlayer.

You can feature as many bust nodes as you want in a VN Scene, place them wherever you want in 2D or 3D space, and you can customize their size or other parameters to your liking (see the Custom VN Scenes section at the end of this guide).

You can also add AnimationPlayers to play various visual effects on bust nodes, such as when an actor speaks, joins or leaves.

Lastly, VN Scenes can be swapped at runtime, allowing you to switch between various layouts.

Note that VN Scenes only handle displaying bust nodes. Spoken text is displayed through the standard UI elements, such as the Dialogue Box, Speech Bubbles or such other methods.

Settings

For VN commands to work, you will need to enable VN Mode. Open **Candy_Engine.gd**, go to the "Settings" region of the script, and look for the VN Mode category:

```
#°############################################################################################
#^
#^                                        SETTINGS
#^
#°############################################################################################
```

```
#& VN MODE
```

Set the **vn_mode** variable to 'true':

```
var vn_mode = true
```

Persistent Busts

If you want busts to persist on screen when dialogue ends, set **persistent_busts_on_end** to 'true'.

```
var persistent_busts_on_end = false
```

Whether you need this option depends on your game's design: you can either make busts persist between dialogues, or you can use a single never-ending dialogue throughout your entire game, using commands to create pauses, hide the dialogue box, display menus and cutscenes, etc.

If you enable persistent busts, you won't be able to use commands to clear busts (i.e. commands only work while a dialogue is running). Instead, you can call the **clear_busts_ood()** function (**Candy_Functions.gd**) to do this. Make sure you pass 'candy_ui' as an argument, or if your game uses multiple engine instances: pass whichever node holds the **Candy_Engine.gd** script relevant to the VN Scene you want to clear (see the Engine Instances guide for more information).

Portraits

You'll probably want to disable speaker portraits, since bust nodes already display illustrations for actors. This is done under the Actor Display category: set **portrait_mode** to "Off" to disable portraits entirely, or to "VN" to only show portraits for off-screen characters (i.e. those who don't have a bust displayed).

```
var portrait_mode = "All"
```

Note that you can also create special rules for specific actors and specific lines (see the **Portraits** guide for more information)

VN Commands

To make a visual novel, you'll use the various VN Commands in conjunction with spoken lines.

§VN_Scene

The §VN_Scene command is used for swapping VN Scenes. Simply provide the name of the .tscn scene file to load.

To remove a VN Scene without replacing it with a new one, leave the Scene field empty.

Note: You should also check the mouse_filter of the default bust nodes: these nodes are provided for demonstrative purposes and you should normally customize the VN Scene (or create your own), but if you use the default VN Scene, you may encounter unexpected behavior when it comes to mouse passing through bust nodes.

You may also want to check the mouse_filter of the root node, to ensure it matches your expectations.

§VN_Bust

When a VN Scene is loaded, all bust nodes are empty: no actors are assigned to them. The §VN_Bust command is used to assign an actor to a node, and simultaneously to select which bust image to display.

Simply provide the following data:

Bust → The name of the bust node to assign an image to.

Reference → The actor or role reference to assign to the bust node.

File → The image file name, or the Sprite Frame resource for AnimatedSprite nodes.

Animation → The name of the animation to play (AnimatedSprite nodes only).

Play/Hold → Whether to play an animation or pause on the first frame (animated images only).

Loop → How many times to play the animation.

Wait → Pause dialogue processing until this number of loops have played.

Time → The time or timestamp until which to pause dialogue.

Time is written in the HH:MM:SS.00 format, or can be written as frames, like so: f=120.

Time is cumulative with Wait. If the Time (or frames) exceeds the length of an animation, the excess carries over into additional loops.

If Wait/Time exceed the number of Loops, dialogue resumes when all loops have played.

You must create a folder for each actor inside **Candy_DE/Media/Characters/Busts**. Image, animation or video files must be placed in each actor's respective folder.

For AnimatedSprite nodes, create a subfolder named "Sprite Frames" in each actor's folder, and save the Sprite Frame resources there.

Once you use the §VN_Bust command, Candy DE remembers which bust node the given actor is assigned to. If you want to change the actor's display image, you can use the §VN_Bust command again, this time without providing a bust node (or you can provide the same node the actor is already assigned to).

§VN_Move

This command can be used when you want to move an actor to another bust node, without interrupting the animation playing on it, and without changing the displayed bust image.

Bust → The name of the new bust node to move an actor to.

Reference → The actor or role reference to assign to the bust node.

Candy DE will continue playing any ongoing animations in the new node, from the same frame it was at when the move occurred, and it will remember how many loops the animation was set to play and how many it has already played.

One important note: the command won't work if the previous bust node and the new bust node are of different types! For example, you can't §VN_Move from a TextureRect to a Sprite2D. This is because different node types have different properties, so settings can't just be ported between them.

With that said, the following node pairs should be compatible:

- TextureRect and NinePatchRect
- Sprite2D and Sprite3D
- AnimatedSprite2D and AnimatedSprite3D

If you need to move a character bust between two different node types, you must use §VN_Bust again.

§VN_Mirror

This command flips a node's image horizontally and/or vertically.

Actors → The bust nodes to flip, separated with a comma (,).

Axis → Whether to flip horizontally, vertically or both.

§VN_Bust_Wait

This command pauses dialogue processing until the animations of the designated bust nodes have played the specified amount of loops and time/frames.

Actors → The bust nodes to wait on, separated with a comma (,).

Wait → Pause dialogue processing until this number of loops have played.

Time → The time amount, timestamp or frame until which to pause dialogue.

See the §VN_Bust command above for details on Wait and Time.

§VN_Bust_Stop

This command stops an animation and optionally reset to the first frame or freezes on the current frame. Simply provide the following data:

Actors → The bust nodes to stop animations on, separated with a comma (,).

Default → -1 = Stop on the last frame, 0 = stop on the current frame. 1+ = Pause on the given frame.

§VN_Remove

This command clears bust nodes, effectively removing any images, animations or videos.

Actors → The bust nodes to remove images/animations from, separated with a comma (,).

§VN_Effect

This command plays visual effects on bust nodes, in the form of animations through an AnimationPlayer.

Actors → The bust nodes to play effects on, separated with a comma (,).

Library → The animation library to load.

Effects → The effects to play

Loop → How many times to play the animation.

Wait → Pause dialogue processing until this number of loops have played.

Time → The time amount, timestamp or frame until which to pause dialogue.

See the §VN_Bust command above for details on Loop, Wait and Time.

Animation Library files must be placed in the **Media/Characters/Busts/Animation_Libraries** folder.

§VN_Effect_Wait

This command pauses dialogue processing until the designated effects, for the designated bust odes, have played the specified amount of loops and time/frames.

Actors → The bust nodes to wait for effects on, separated with a comma (,).

Loop → How many times to play the effect animation.

Wait → Pause dialogue processing until this number of loops have played.

Time → The time amount, timestamp or frame until which to pause dialogue.

See the §VN_Bust command above for details on Loop, Wait and Time.

§VN_Effect_Stop

This command stops the specified effects for the specified bust nodes.

Actors → The bust nodes to stop effects on, separated with a comma (,).

Effects → The specific effects to stop.

Static Busts

If you only plan to use static images for busts, you can display them inside TextureRect, NinePatchRect or TextureButton nodes.

Simply add the image files to each character's Busts folder, like so:

Your Project/Candy_DE/Media/Characters/Busts/Alice

You can also display static busts in Sprite2D/3D, AnimatedSprite2D/3D or VideoStreamPlayer nodes: just set the Loop value in the §VN_Bust command to 0, and it will pause on the first frame. See the 'Background Animations' section below for details on these nodes.

TextureButton

TextureButton nodes can accept multiple textures, in addition to texture_normal: texture_pressed, texture_hover, etc. Candy DE is designed to handle this automatically if you have corresponding files:

If the bust display node is a TextureButton, a subfolder named "Button Textures" can be created inside the actor's busts folder. Inside that folder, you can create a subfolder for each normal texture file: name these subfolders exactly as the normal texture file they match to. Use underscore (_) instead of a coma (.) to separate the file extension in the subfolder name.

For example, if the file you plan to assign to texture_normal is named "Happy.png", name the subfolder "Happy_png".

In each subfolder, you can add files for the various texture types that TextureButton nodes support: texture_pressed, texture_hover, texture_disabled, texture_focused and texture_click_mask.

Name these files anything you like, but add the texture type as a prefix. For example: "texture_pressed.png" or "Happy_texture_pressed.png" will both work.

Candy DE will automatically assign the files to the TextureButton node if they exist (having files for all texture types is not required).

To be clear: the file for texture_normal must be placed in the actor's folder (e.g. Candy_DE/Media/Characters/Busts/Alice). It does not need the texture_normal suffix in its name (e.g. "Happy.png" is fine). This allows you to re-use the same file for other node types.

Bust Animations

If you want busts to play animations, you have three options: Sprite nodes, AnimatedSprite nodes, or VideoStreamPlayer nodes.

For the first two, you should specify the play speed (as frames per second) in **Bust_Scene.gd**:

```
@export var sprite_fps = 30.0
```

There is no option to set a custom frame rate for individual animations, except to change the **sprite_fps** variable at runtime with a §Set command before using the §VN_Bust command. Therefore, it's recommended that you design all animations for the same frame rate.

Sprite2D/3D

Sprite nodes require a sprite sheet in order to play a bust animation. This is standard: it should be a single image file, split in a grid, where each cell represents an animation frame.

The file name must end with a special tag that will tell Candy DE how many rows and columns the sprite sheet has. For example:

Smile_4x6-2.png

The tag is the blue part: '_4x6-2' (or any other numbers): 4 is the number of rows (H Frames), 6 is the number of columns (V Frames), and 2 is the number of 'missing' (empty) frames in the last row. If no frames are missing, writing '-0' is optional.

The file format (.png) can be anything you want as long as it's supported by Sprite nodes, and "Smile" can be replaced with any text you want (e.g. "Frown", "Jump", etc.).

This means you'll need to take care to add the tag in sprite sheet names when creating and saving them, but no extra work will be required beyond that to make Candy DE use them correctly.

AnimatedSprite2D/3D

If you use AnimatedSprite nodes, you'll need to save SpriteFrames resources (.tres).

Unlike other bust files, those SpriteFrames resources must be placed in their own subfolder, named "Sprite Frames", and located inside a character's Busts folder. For example:

Your Project/Candy_DE/Media/Characters/Busts/Alice/Sprite Frames

You can create as many Sprite Frame resources as you like for each character, although in most cases one per character is enough.

Make sure your Sprite Frame resources have the correct animations configured, and Candy DE will display them properly if you correctly provide the §VN_Bust command with a Sprite Frame resource file name and the name of the animation track to play.

VideoStreamPlayer

Although this is unconventional, you are free to use VideoStreamPlayer nodes to display busts. Simply save the video files inside the corresponding character's Busts folder, then setup the §VN_Bust command correctly.

Animated Folders

Note: Animated Folders only work with TextureRect and NinePatchRect nodes. TextureButton nodes are not supported at this time.

You have the option of creating animated VN Busts from multiple image files, which act as frames for the animation. To do this, follow these steps:

1. Inside an Actor's busts folder, create a folder called "Animated". For example: **Candy_DE/Media/Characters/Busts/Alice/Animated**
2. In the Animated folder, create a folder for each animation. Give these folders any name you want.
3. Place the image files for each animation inside their corresponding folders.
4. The files will be displayed in alphabetical order, so name them appropriately (e.g. Smiling_001, Smiling_002, etc.).

Make sure to use zero-padding for numbers: this means writing '001' instead of just '1'. Otherwise, '10' will come before '2' or '9' in alphabetical order.

Be careful with animation folder names that match standalone image or video files: when you provide a file name without an extension to a command (e.g. "Smiling" instead of "Smiling .png"), Candy DE checks if an animation folder of that name exists. If so, it uses that animation.

If no animation folder of the same name exists, Candy DE checks if a file of the same name, with the default extension, exists (e.g. "Smiling .png" if .png is set as the default).

In other words, if you have an animation folder named "Smiling", you can't provide "Smiling" as a substitute for "Smiling.png": Candy DE will use the "Smiling" animation.

Optionally, in each animation folder, you can create a txt file named 'config.txt'. In that file, add this line:

fps = 2.0

This tells the dialogue engine the speed to play each animation at, in frames per second. For example, a value of 5 means that 5 images will be displayed per second.

You can replace the fps value with any number you want, including decimal numbers.

If an animation doesn't have a config.txt file, the engine will use the default **image_fps** value of the portrait box (this is an exported variable in the **Bust_Scene.gd** script; you can modify it in the inspector, in the portrait box scene: **Candy_DE/Scenes/VN Scenese/Default.tscn**).

Finally, to make Bust nodes display an animation, in the §VN_Bust command provide the name of the animation's folder as the file to assign to a Bust node.

Join/Move/Leave Effects

Instead of making bust images suddenly appear or disappear, you have the option of playing a visual effect such as sliding, fading or any other animation you want to configure.

By default, you have the option to use an AnimationPlayer to play an animation of your choice. However, you can code other methods if you wish to.

For the AnimationPlayer method, you must give each bust a child AnimationPlayer with the name "JML", and create the animations you want to use as effects.

You must then indicate which animations to use. This can be done in three places:

In the **Candy_Engine.gd** Settings:

```
@export var vn_join_effect = ["", ""]
@export var vn_move_from_effect = ["", ""]
@export var vn_move_to_effect = ["", ""]
@export var vn_leave_effect = ["", ""]
```

In the **Bust_Scene.gd** script (overrides the **Candy_Engine.gd** settings):

```
@export var vn_join_effect = ""
@export var vn_move_from_effect = ""
@export var vn_move_to_effect = ""
@export var vn_leave_effect = ""
```

In the **actors** dictionary in **Candy_Database.gd** (overrides both of the above):

```
"VN Join Effect": ["", ""],
"VN Move From Effect": ["", ""],
"VN Move To Effect": ["", ""],
"VN Leave Effect": ["", ""],
```

In the array, provide a method and a value, like so: ["Method", "Value"]. For the JML player, the method is "Animation", and the value is the name of whatever animation you want to play.

If no method is provided, no JML effect will play.

Note that dialogue processing will be paused until the effects are finished playing.

Speaker Highlighting

You'll probably want to draw attention to the bust node of whichever actor is speaking.

This can be done using the **highlight_speaker()** function, along with its **end_highlight_speaker()** counterpart, and the **speaker_highlight** dictionary (all located in the **Bust_Scene.gd** script).

The **speaker_highlight** dictionary simply contains various parameters that determine what methods are used to highlight the speaker's bust node.

The **highlight_speaker()** function is called automatically by Candy DE when processing a spoken line, if VN Mode is enabled. Inside the function, for each dialogue mode, you can write code that determines what should happen when the various highlighting options are enabled.

The **end_highlight_speaker()** function is also called automatically, when a character is finished speaking. It undoes the highlighting effects.

You can code your own effects in any way you want. To make things easier, a few basic effects are already provided:

NameTag

This option displays the speaker's name near their bust. You'll need to add a child Label or RichTextLabel node to each bust node, and place it wherever you want on screen.

By default, the speaker's name is displayed without any BBCode formatting. An alternate line of code is provided if you want to use the same BBCode formatting as when the speaker name is displayed in the dialogue box.

```
#% Write speaker name in NameTag:
#+ No BBCode:
speaker_bust.get_node("NameTag").text = candy_de.actors[speaker_ref]["Display Name"]
#+ Alternative - copy text from dialogue box speaker field to replicate BBCode formating:
#speaker_bust.get_node("NameTag").text = get_node(candy_ui.dialogue_box_path).speaker_node.text
```

Note that if you use other modes of displaying text (e.g. Subtitles or Speech Bubbles) you'll have to get creative in order to copy the speaker name's BBCode formatting, since those modes don't have a dedicated node for displaying the speaker name.

Outline

The outline option is used if you want to draw an outline around the speaker bust. This is done by overlaying an outline image over the bust image. You'll need to do a few things to set this up:

1) Add a child TextureRect, NinePatchRect, TextureButton, Sprite2D/3D or AnimatedSprite2D/3D to each bust node, and name these children "Outline".
2) Create outline image files, which should be transparent except for the drawn outline.
3) Write your own code to display the outline images.

This can be a bit of work, but shouldn't be too hard. Obviously, if you intend to use animated outlines to match bust animations, it will be a bit more complex to implement, but static outlines should be straightforward.

Scale

The Scale option changes the size of the speaker's bust node slightly (by default: increased by 10%).

In the **speaker_highlight** dictionary, just set **"Scale"** to 'true' and **"Scale_Size"** to whatever value the bust node's scale should be multiplied by.

Animation_1

This option plays an animation of your choosing, either on the bust node directly or perhaps on some other node to achieve various effects.

You'll need to add an AnimationPlayer node to the Bust Scene, but **not as children of bust nodes**. You'll also need to provide the path to that AnimationPlayer in the **highlight_player_1** variable.

As a suggestion, you can create animations that brighten the speaker's bust node, make it shake or hop, alter its saturation, or anything else you might imagine.

If you want to play multiple animations on the speaker, you can add more AnimationPlayer nodes, more Animation keys to the **speaker_highlight** dictionary, and more Animation conditions in **highlight_speaker()** -- e.g. "Animation_2", "Animation_3", etc.

Billboard

The Billboard option is only used with Sprite3D and AnimatedSprite3D bust nodes. It changes the Billboard mode on the bust nodes (typically to make them face the camera, assuming that 3D VN nodes don't always face the camera in your game).

To choose which mode the Billboard property is changed to, edit the values of the **billboard_on** and **billboard_off** variables:

```
@export var billboard_on = BaseMaterial3D.BILLBOARD_ENABLED
@export var billboard_off = BaseMaterial3D.BILLBOARD_DISABLED
```

Final Notes:

- As mentioned previously, you can create your own options entirely, or modify the existing ones.
- Remember to also edit **end_highlight_speaker()** to undo the effects of **highlight_speaker()** once the speaker has finished speaking.

- By default, in both functions, code is only provided for the "Box" dialogue mode. This is just for simplicity: if you use other dialogue modes, you can copy/paste the default "Box" code under the other modes.

Speech Bubbles

If you want to display spoken text in speech bubbles in VN Mode, you must add a speech bubble as a child to each bust node. See the **VN Scene Customization** guide for details.

You also need to set **dialogue_mode** to "Bubbles" in **Candy_Engine.gd**, under the Settings section.

If an actor isn't assigned to a bust node when speaking, Candy DE will use whatever alternate dialogue mode you have chosen for **bubble_exempt_mode** in **Candy_Engine.gd**:

```
var bubble_exempt_mode = "Box"
```